Chauncey Thompson

Intro & Hypothesis

The goal of this project was to analyze the performance of the computer's memory hierarchy, including the L1, L2, and L3 caches and main memory (DRAM). The experiments examined how memory access patterns, working-set sizes, read/write operations, and system load affect memory latency and bandwidth.

I expected smaller working sets to experience lower latency because they could fit within the CPU caches. I also expected sequential access patterns to perform better than random accesses because of hardware prefetching and better spatial locality. Finally, I expected memory latency to increase as memory bandwidth approached saturation.

Hardware and Software Environment

The experiments were conducted using the following hardware and software:

Component	**Specification**
CPU	Intel Core i5-14400F
CPU cores / threads	10 cores / 16 threads
L2 cache	9.5 MiB total
L3 cache	20 MiB
RAM	32 GB DDR5-4800
Operating system	Windows 11
Compiler	GCC 16.2.0
Memory benchmarking	Intel Memory Latency Checker v3.13
Hardware profiling	Intel VTune Profiler 2026.4
Analysis and plotting	Python and Matplotlib

Intel MLC was used to measure memory latency and bandwidth. Intel VTune was used to analyze cache misses, TLB misses, and memory-related CPU stalls.

Chauncey Thompson

Methodology

The experiments measured memory performance under different access patterns and working-set sizes. Intel MLC was used for the latency, bandwidth, and throughput experiments, while a custom C benchmark was used for the cache and TLB analysis.

The main measurement procedures were:
Cache latency: Five trials per memory level using random-access latency measurements on CPU 0.
Access patterns: Three trials each for sequential and random memory accesses at 64-, 256-, and 1024-byte strides.
Bandwidth: Five trials for the available read/write traffic configurations.
Loaded latency: Five trials using seven bandwidth-generating threads and multiple injection delays.
Working-set size: Three trials for each buffer size, ranging from 16 KiB to 512 MiB.
Cache/TLB profiling: Intel VTune measured hardware performance counters while a custom C program repeatedly accessed allocated memory.

Results from repeated Intel MLC experiments were averaged to reduce the influence of measurement variation. The raw results were saved in “data/raw”, and graphs were generated using Python scripts. The C benchmark was compiled with GCC using the “-O2” optimization level. Its output included a checksum to verify that the memory accesses were executed. The experiments were run on Windows without a controlled CPU-frequency governor. CPU frequency changes, scheduling, background processes, and thread migration could introduce additional measurement variation.

Cache Hierarchy and DRAM Latency

The first experiment measured latency at different levels of the memory hierarchy using buffer sizes intended to target each cache level or main memory.

Memory level	Average latency
L1 cache	1.12 ns
L2 cache	3.52 ns
L3 cache	14.02 ns
DRAM	83.84 ns

The results show that accessing smaller, cache-resident working sets is much faster than accessing main memory. L1 cache had the lowest measured latency at about 1.12 ns, while

Chauncey Thompson
October 6, 2026

DRAM had the highest at 83.84 ns. This is consistent with my original hypothesis. The measured differences occur because caches are physically closer to the CPU and are designed to provide faster access to recently used data.

Sequential and Random Access Performance

Memory access patterns were tested using 64-, 256-, and 1024-byte strides.

Bandwidth results:

Stride	Sequential bandwidth	Random bandwidth
64 B	25.77 GB/s	23.66 GB/s
256 B	12.05 GB/s	5.66 GB/s
1024 B	9.65 GB/s	5.60 GB/s

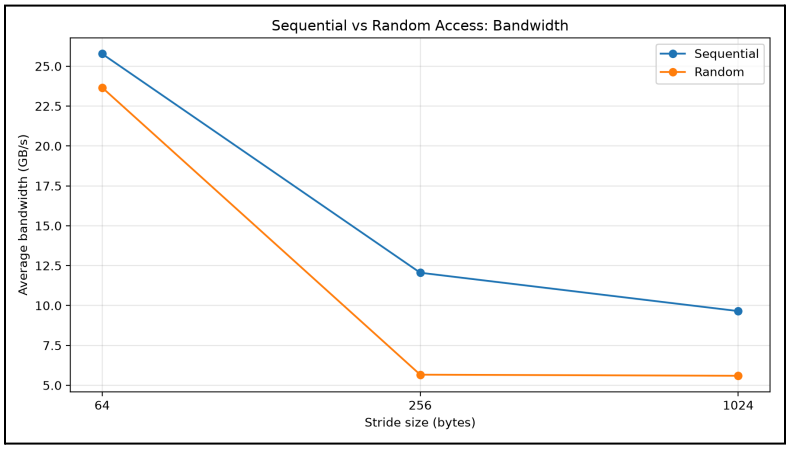


Figure 1: Sequential vs random access: Bandwidth

Sequential access generally achieved higher bandwidth than random access, especially at larger stride sizes. At a 256-byte stride, sequential access achieved approximately 12.05 GB/s, compared to only 5.66 GB/s for random access. This difference shows the importance of spatial locality and hardware prefetching.

Latency results:

Stride	Sequential latency	Random latency
64 B	3.73 ns	29.30 ns
256 B	7.53 ns	90.17 ns
1024 B	8.80 ns	91.07 ns

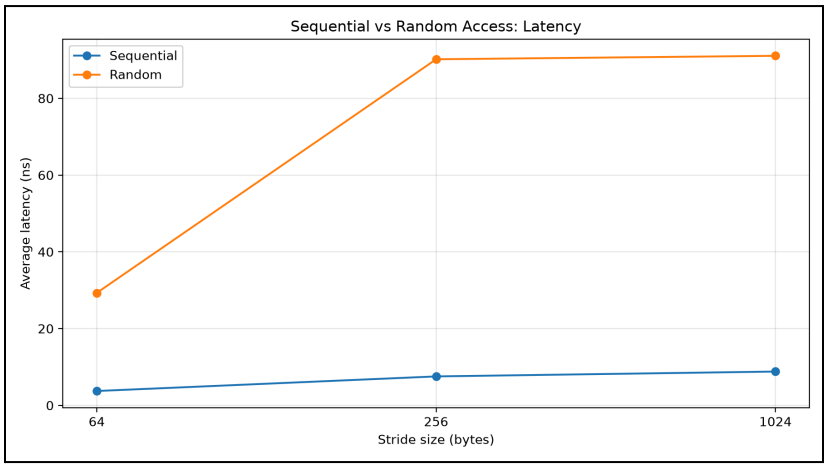


Figure 2: Sequential vs random access: Latency

Random accesses generally produced much higher measured latency than sequential accesses. At a 256-byte stride, random-access latency reached about 90.17 ns, compared with 7.53 ns for sequential access. These measurements show that memory-access organization significantly influences performance.

Chauncey Thompson

Read/Write Traffic Mixes

Different combinations of memory reads and writes were tested to determine their effects on bandwidth.

Single-thread traffic measurements:

Configuration	Average bandwidth
100% read	26.09 GB/s
100% non-temporal write	50.86 GB/s
50% read / 50% write	33.26 GB/s

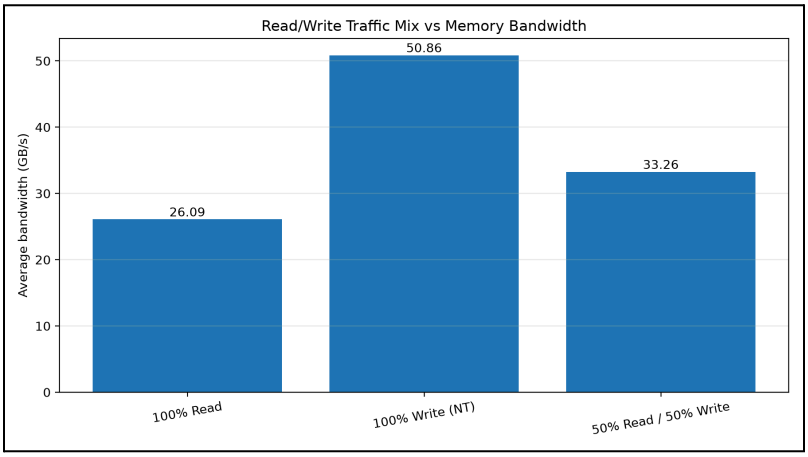


Figure 3: Read/Write Traffic vs memory bandwidth

The non-temporal write configuration achieved the highest bandwidth in this experiment. Non-temporal stores use a different cache policy from ordinary writes and can avoid some cache-related overhead. Therefore, the write-only result is not directly comparable with ordinary cached writes.

System-wide peak bandwidth:

The peak-bandwidth experiment also tested different read/write ratios using multiple threads.

Configuration	Average bandwidth
100% read	65.54 GB/s
75% read / 25% write	62.85 GB/s
Approximately 67% read / 33% write	62.36 GB/s
50% read / 50% write	62.11 GB/s
Triad workload	62.83 GB/s

The highest measured bandwidth was approximately 65.54 GB/s for the read-only configuration. These results show that the traffic make-up affects possible bandwidth, although the overall differences between the system-wide mixed workloads were relatively small. The system-wide measurements are not directly interchangeable with the single-thread measurements because the

number of memory-generating threads and workload configurations are different. An exact 70/30 read/write mix was not directly measured so the available 75/25 and approximately 67/33 mixes were used as nearby alternatives.

Throughput-Latency Relationship and Little's Law

This experiment investigated the relationship between memory bandwidth and access latency under increasing system load. Seven bandwidth-generating threads were used to increase memory pressure. Injection delays were adjusted to produce different bandwidth levels.

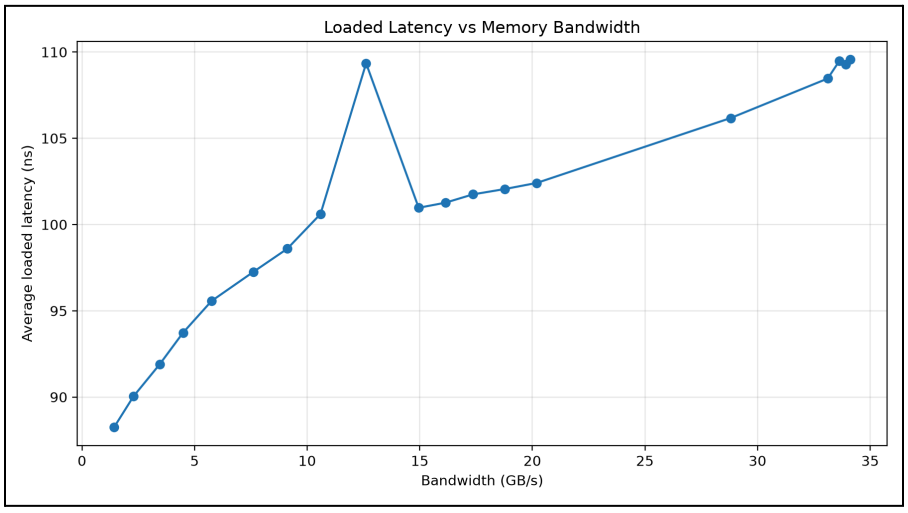


Figure 4: Loaded latency vs memory bandwidth

At relatively low bandwidth, measured memory latency was approximately 90 ns. As bandwidth increased, latency generally rose. Near saturation, bandwidth approached approximately 34 GB/s while loaded latency was around 109 ns. A high-load point occurred at approximately 33.12 GB/s and 108.46 ns. The curve indicates a saturation region around 33–34 GB/s for this specific seven-thread random-access workload. Near this region, additional offered load produced relatively little increase in bandwidth. This is the memory-system saturation knee for the tested workload, not necessarily the maximum bandwidth of the entire system.

Little's Law

Little's Law states: $N = \lambda W$, where:

- N represents the average number of operations in the system
- λ represents the operation throughput
- W represents the average time an operation spends in the system

Applied to memory systems, the relationship explains why more outstanding memory requests are needed to maintain higher throughput when individual accesses have significant latency. As the memory subsystem becomes saturated, requests may spend more time waiting for service.

Chauncey Thompson

Latency can then rise even when bandwidth stops increasing significantly. The observed throughput-latency curve is consistent with this relationship. A precise estimate of outstanding memory transactions would require appropriately matched transaction-throughput and latency measurements.

Working-Set Size and Cache Transitions

This experiment tested memory buffer sizes ranging from 16 KiB to 512 MiB. The measurements showed several noticeable latency increases as working-set size increased.

Working-set size	Average latency
16 KiB	1.10 ns
64 KiB	3.30 ns
512 KiB	3.73 ns
1 MiB	4.40 ns
2 MiB	14.77 ns
16 MiB	23.30 ns
32 MiB	73.27 ns
128 MiB	90.10 ns
512 MiB	91.60 ns

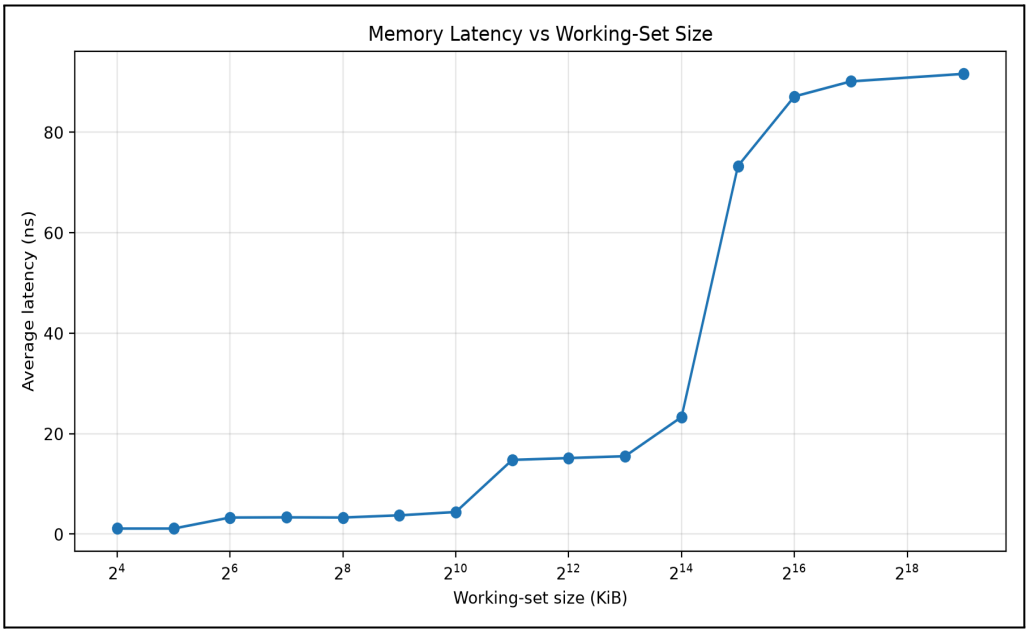


Figure 5: Memory latency vs Working set size

The first significant increase appeared between approximately 32 KiB and 64 KiB, consistent with the working set exceeding the effective L1 cache capacity. Another noticeable increase appeared between 1 MiB and 2 MiB, implying a transition away from L1/L2 cache access. A larger increase occurred between 16 MiB and 32 MiB, consistent with the working set exceeding the system's 20 MiB shared L3 cache and relying more heavily on DRAM. These boundaries are approximate because measured latency can also be influenced by cache associativity, prefetching, memory placement, and the processor's heterogeneous core architecture. Overall, the results demonstrate that memory latency depends strongly on whether the active working set fits within the cache hierarchy.

Chauncey Thompson

Cache-Miss and TLB-Miss Analysis

A custom C benchmark was analyzed with Intel VTune to measure cache misses, TLB misses, and CPU memory stalls. The benchmark used 10,000 repetitions to provide sufficient execution time for sampling.

Two configurations were tested:

Metric	4 MiB / 64 B	128 MiB / 4096 B
Runtime	1.004 s	4.050 s
Memory Bound	49.8%	87.6%
DRAM Bound	0.3%	88.8%
Reported LLC misses	0	296,024,278
Completed 4 KB page walks	0	302,509,075
Retired loads missing STLB	0	304,709,141

The smaller workload had relatively low DRAM-related stalls because its working set fit within the shared L3 cache. Repeated accesses could generally be served without frequent main-memory requests. The larger workload was largely more DRAM-bound, with VTune reporting approximately 296 million LLC misses.

TLB behavior

The TLB stores cached virtual-to-physical address translations. With a 4096-byte stride, successive accesses generally reach different 4 KiB pages. This can increase pressure on the TLB and start additional page walks. VTune reported about 302.5 million completed 4 KiB page walks for the larger workload, compared with zero reported for the smaller workload.

The larger workload also required more time to execute, supporting the observation that less favorable memory locality is associated with greater memory-system overhead. The two configurations differ in working-set size, stride, and total memory-access count. In turn, their runtime difference cannot be linked only to either TLB misses or cache misses.

The zero values are reported hardware-counter estimates, not proof that no misses happened. The hardware-profiling comparison was collected once per configuration, so the results should be considered representatively rather than a statistically repeated comparison.

Chauncey Thompson

Large-page limitation

A large-page comparison was considered but wasn't carried out. My windows account did not list the "SeLockMemoryPrivilege" privilege, which is normally required for allocating large pages. So because large-page support was unavailable under the account configuration, the experiments used ordinary memory pages.

Benchmark Pitfall

One important benchmarking mistake is interpreting sequential-access latency as true DRAM latency. During the preliminary measurements, a large sequentially accessed buffer caused an apparent latency of approximately 3.6 ns, despite being much larger than the processor's caches. This result was misleading because hardware prefetching allowed the CPU to retrieve data before it was explicitly requested. After changing to a random-access measurement, the apparent DRAM latency increased to approximately 83 ns, which was much more consistent with main-memory access. This experiment demonstrates that a benchmark can produce technically valid numerical output while measuring something different from what an experimenter would intend. To avoid misleading conclusions, I used random-access measurements for the primary cache-hierarchy latency analysis and documented the effects of prefetching on sequential-access results.

Experimental Limitations and Reproducibility

Several factors could influence the results:

CPU frequency and scheduling: Windows background activity and dynamic CPU frequencies can affect timing consistency.

Prefetching: Sequential-access measurements can report prefetched cache-access latency rather than actual DRAM latency.

Cache targeting: Buffer-size experiments approximate cache levels but do not perfectly isolate each cache.

Hardware counters: VTune counter values may be sampled or scaled estimates, and some measurements were collected only once.

Traffic configuration: Non-temporal writes behave differently from ordinary cached writes.

Large pages: A controlled large-page experiment was not available under the tested Windows account.

NUMA and affinity: The project did not perform a controlled NUMA-placement comparison. CPU 0 was selected for certain latency measurements, while other workloads used different thread configurations.

Conclusion

This project demonstrated how cache hierarchy organization, memory access patterns, and system load influence memory performance. The L1 cache provided the lowest measured latency at approximately 1.12 ns, while DRAM latency was approximately 83.84 ns.

Chauncey Thompson

Sequential memory access generally achieved better bandwidth and lower measured latency than random access, particularly at larger stride sizes. The throughput-latency experiment identified a saturation region around 34 GB/s for the tested seven-thread random-access workload.

Working-set measurements showed noticeable performance transitions as buffer sizes exceeded effective cache capacities. Finally, Intel VTune demonstrated that the larger, page-strided benchmark experienced substantial LLC misses, TLB page walks, and DRAM-related stalls. Overall, the experiments highlight the importance of cache locality, memory access patterns, and careful benchmark design when analyzing computer-system performance.

AI-Use Disclosure

ChatGPT was used to assist with understanding the assignment requirements, explaining benchmarking tools, troubleshooting commands and scripts, interpreting measurements, generating plotting code, and organizing the report structure.